



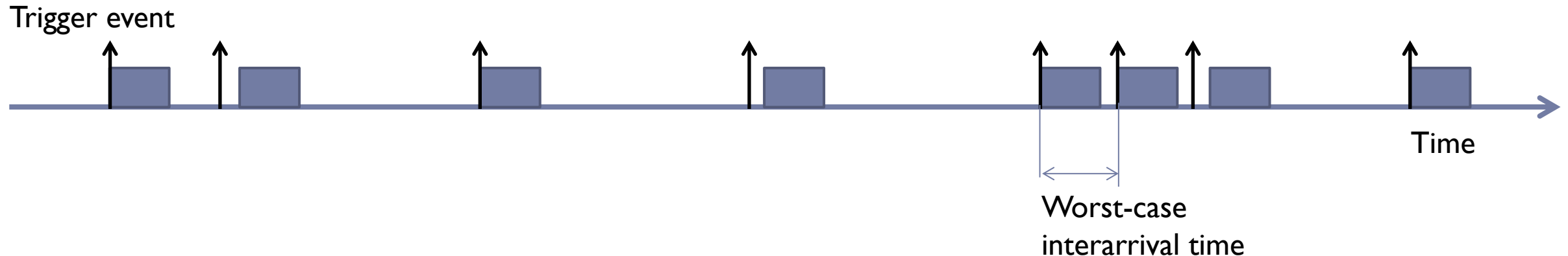
Servers



Luís Miguel Pinho

Supporting Sporadic Tasks

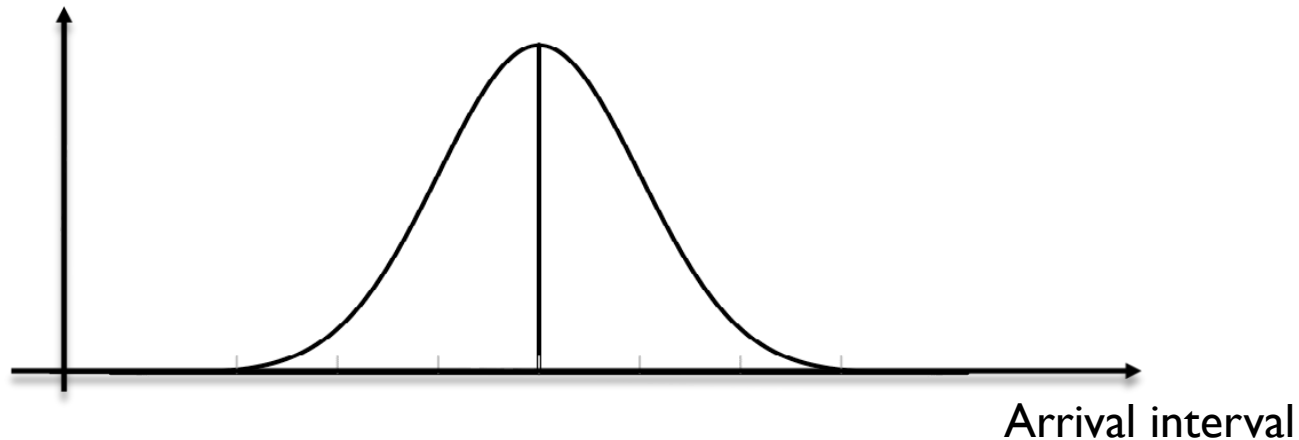
- ▶ Remembering tasks can be non-periodic (i.e., event-driven)
 - ▶ Sporadic activities have a maximum frequency of repetition
 - ▶ Available from the specification



- ▶ We can include sporadic tasks in the usual periodic model
 - ▶ Highly pessimistic

Supporting Aperiodic Tasks

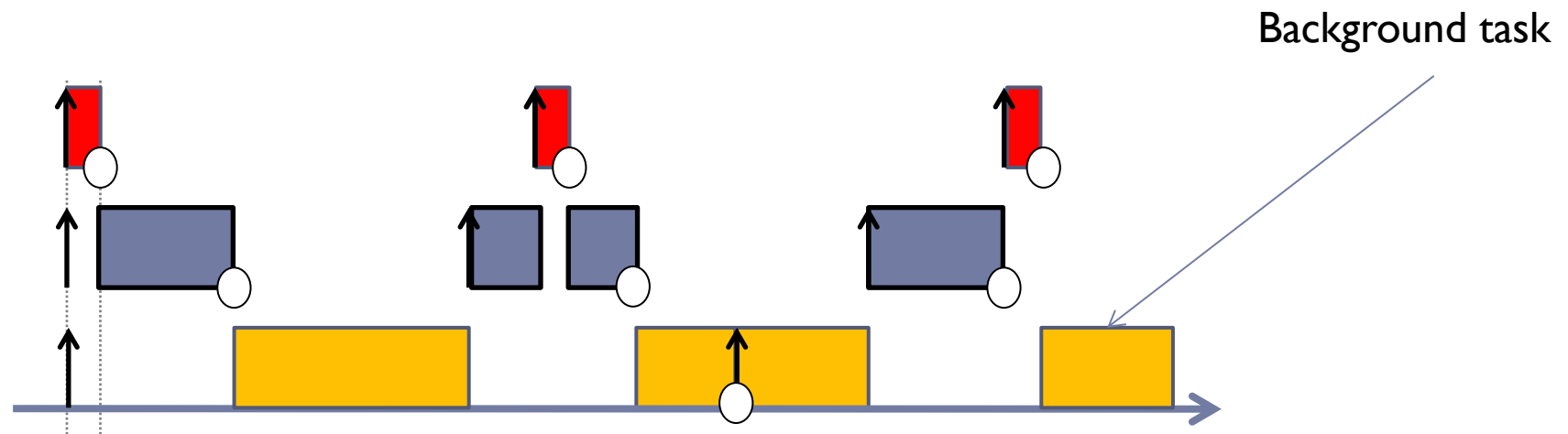
- ▶ Remembering tasks can be non-periodic (i.e., event-driven)
 - ▶ Aperiodic activities have no defined periodic information
 - ▶ Potentially only stochastic characterization



- ▶ Usually systems combine different types, how to schedule aperiodic safely?
 - ▶ Limit impact (interference) on periodic tasks, but still provide some Quality of Service (QoS)

Supporting Aperiodic Tasks

- ▶ One approach is to execute in background
 - ▶ Give them the lowest priority or “infinite” deadline
 - ▶ Aperiodic tasks only execute “best effort”, if other tasks are not executing



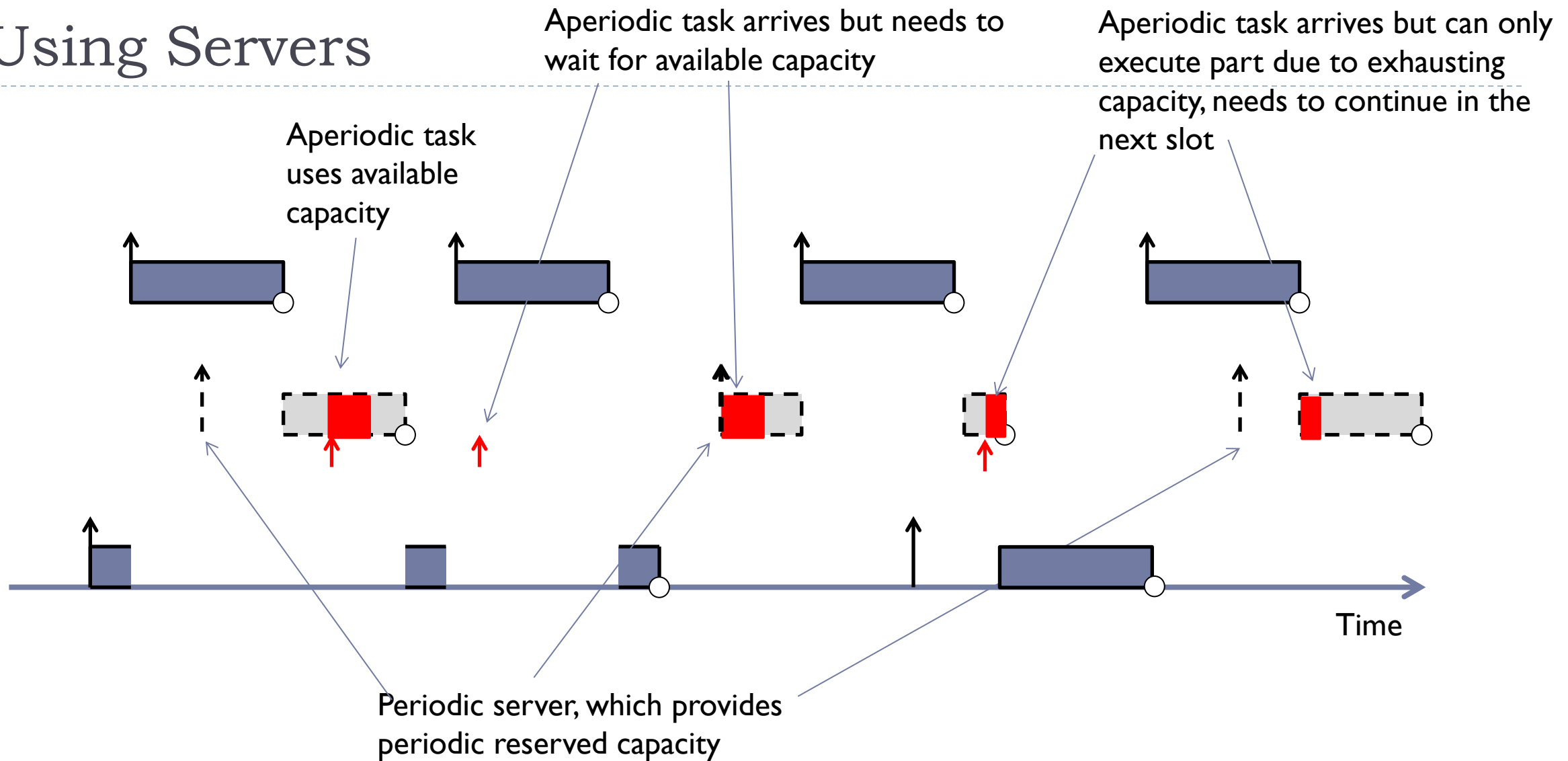
Supporting Aperiodic Tasks

- ▶ One approach is to execute in background
 - ▶ Give them the lowest priority or “infinite” deadline
 - ▶ Aperiodic tasks only execute “best effort”, if other tasks are not executing
- ▶ Easy to implement and does not interfere with periodic tasks
 - ▶ Small overhead only of handling the trigger event (interrupts)
- ▶ But can suffer large service delays, with low QoS
 - ▶ QoS can be calculated
- ▶ Poor performance for real-time aperiodic tasks (e.g., alarms), suitable for non-real-time tasks (e.g. logging)

Using Servers

- ▶ An approach to support aperiodic is to use servers
 - ▶ A special task whose purpose is just to execute active aperiodic tasks
 - ▶ This task is called aperiodic tasks server and it is characterized by a period T_S and a capacity C_S
 - ▶ This way, it is possible to handle the server as if it was a periodic task with the required priority/deadline, and obtain the desired QoS

Using Servers

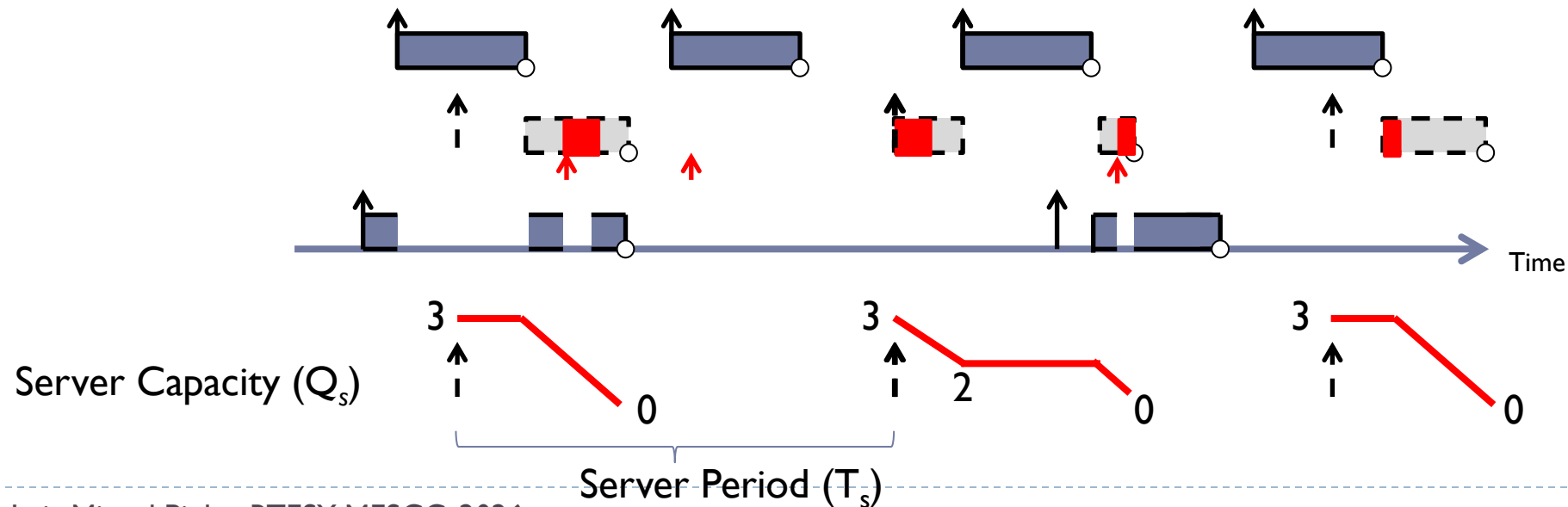


Using Servers

- ▶ There are various types of servers designed to handle aperiodic tasks, which can operate with either fixed or dynamic priorities. These differ in several aspects:
 - ▶ Impact on the schedulability of the periodic tasks
 - ▶ Average response time of aperiodic tasks
 - ▶ Overheads and implementation cost
- ▶ Fixed priority servers include:
 - ▶ Polling server
 - ▶ Deferrable server
 - ▶ Sporadic server
- ▶ Dynamic priority servers include:
 - ▶ Total Bandwidth Server (TBS)
 - ▶ Constant Bandwidth Server (CBS)
- ▶ Others exist, which we will not consider in RTAES

Polling Server

- ▶ Equivalent to executing a periodic task with T_s period and Q_s execution time (denoted capacity)
 - ▶ Aperiodic requests are executed during the time assigned by the scheduler
 - ▶ Even if no requests, the server capacity is used
 - ▶ The capacity is replenished every period



Polling Server

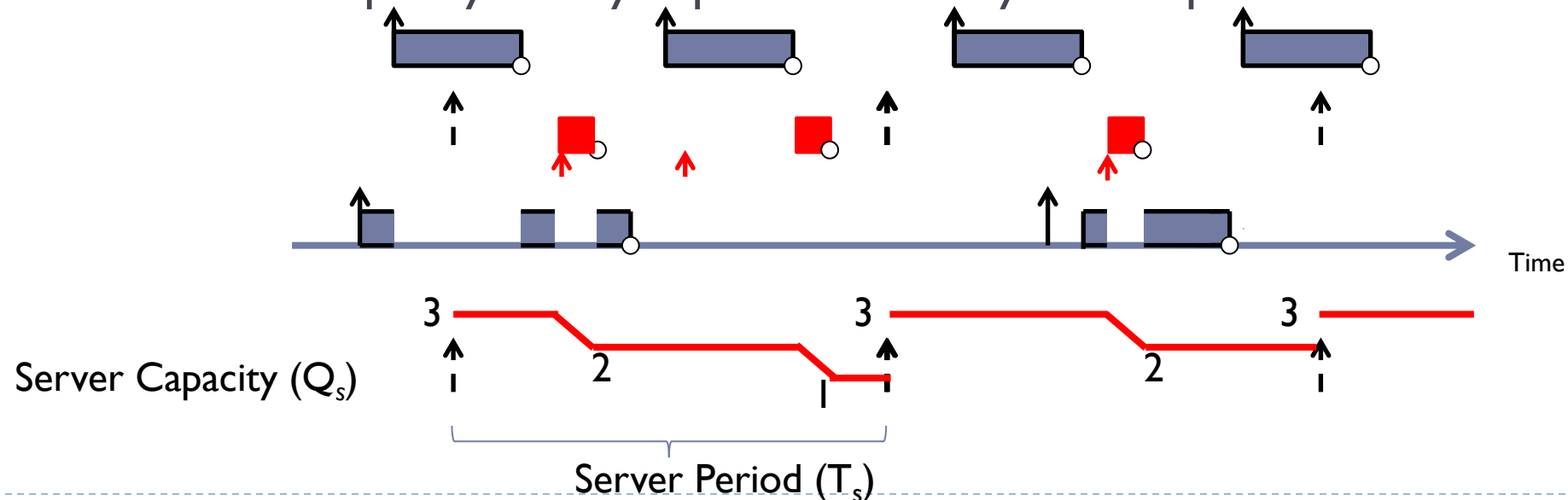
- ▶ Equivalent to executing a periodic task with T_s period and Q_s execution time (denoted capacity)
 - ▶ A polling server is easy to implement, with just a single queue for aperiodic requests and monitoring the capacity being used
 - ▶ The average response time for aperiodic requests is better compared to background execution because these tasks can run at a higher priority level
 - ▶ However, there are still periods of unavailability because of the server's period
 - ▶ Effect on schedulability of the periodic taskset is similar to that of the corresponding virtual periodic task

$$U_p + U_s \leq (n + s)(2^{1/(n+s)} - 1)$$

- ▶ Where n is the number of periodic tasks, s the number of servers, U_p is the total utilization of periodic tasks, U_s is the total servers utilization:
- ▶ Replacing servers for an equivalent periodic task in the schedulability analysis, allows to use the usual limit (n is the total number of tasks, including servers): $U \leq n(2^{1/n} - 1)$

Deferrable Server

- ▶ Can defer capacity to when it is needed - simulate interrupt handling but bounds the available time for aperiodics
 - ▶ The server keeps its capacity when not used
 - ▶ Therefore, an aperiodic request can be immediately executed if the server is active, still has capacity, and no high priority task is executing
 - ▶ The server capacity is fully replenished every server period



Deferrable Server

- ▶ Can defer capacity to when it is needed - simulate interrupt handling but bounds the available time for aperiodics
 - ▶ Implementation is similar to polling server
 - ▶ Response time to the aperiodic requests is improved in average (compared to polling server) because it is possible to use the capacity during the period, thus reducing unavailability periods
 - ▶ However, it has a higher impact in lower priority tasks, as it may concentrate interference increasing lower priority tasks response time
 - ▶ Delayed executions increase the load on the future, and it is possible having two consecutive executions (back-to-back execution)
 - ▶ E.g., using RM+DS and a server with highest priority:

$$U_p + U_s \leq U_s + n \left(\left(\frac{U_s + 2}{2 * U_s + 1} \right)^{1/n} - 1 \right)$$

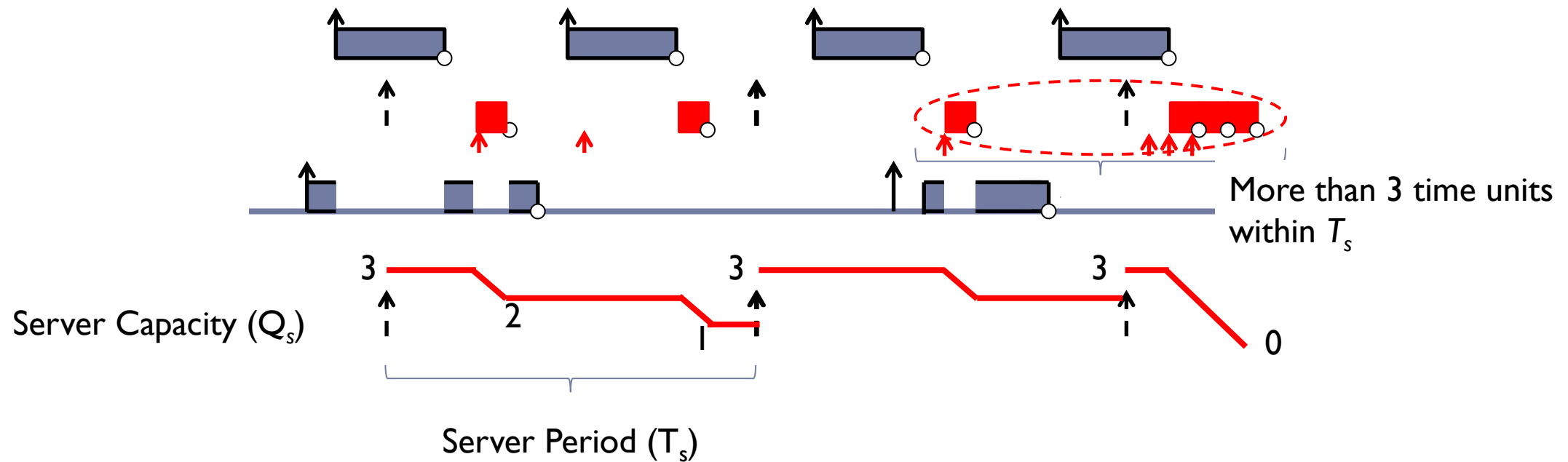
Sporadic Server

▶ Bandwidth preserving servers

- ▶ A deferrable server may delay lower-priority tasks for more time than a period task with the same period and execution time.
 - ▶ Sporadic servers are designed to improve over a deferrable server in this respect.
- ▶ The consumption and replenishment rules of sporadic server algorithms ensure that each sporadic server with period T_s and budget Q_s never demands more processor time than the periodic task (T_s, Q_s) in any time interval
- ▶ Consequently, we can treat the sporadic server exactly like the periodic task
 - ▶ A system of periodic tasks containing a sporadic server may be schedulable while the same system containing a deferrable server with the same parameters is not

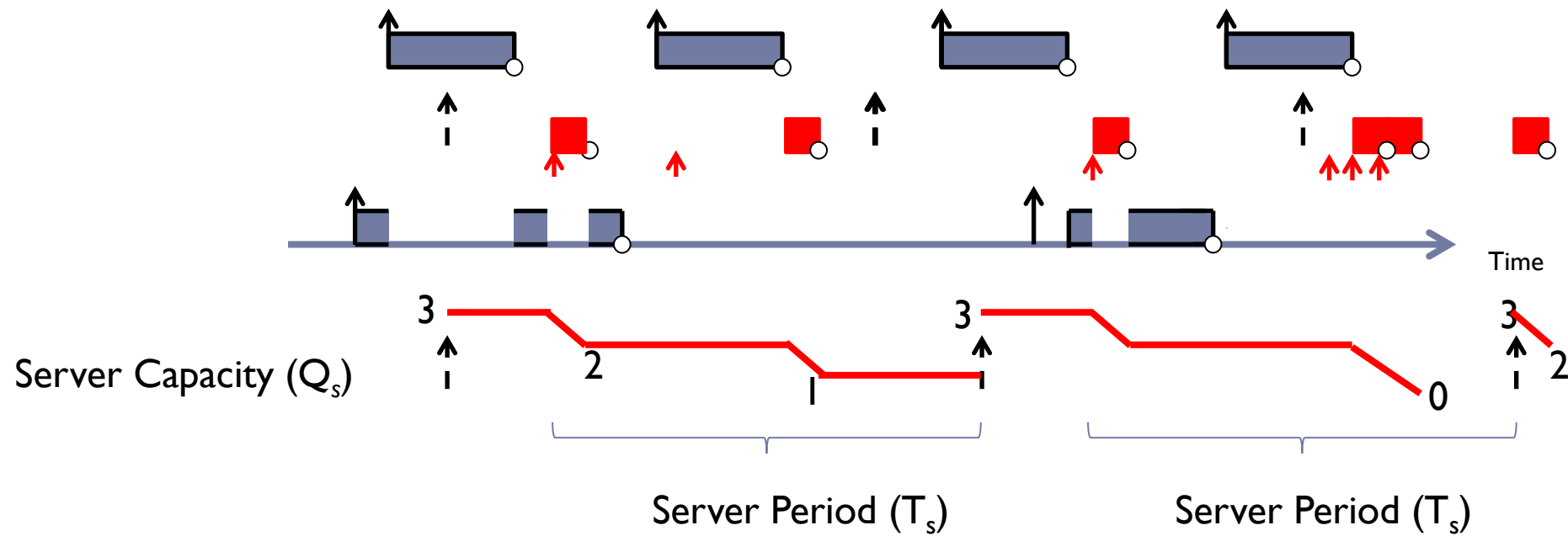
Sporadic Server

► Revisiting the Deferrable Server



Sporadic Server

- Sporadic Server also defers replenishment



Sporadic Server

- ▶ **Sporadic Server also defers replenishment**
 - ▶ Complexity of implementation is higher as we need to maintain the info of the replenishment instants and the amounts to replenish
 - ▶ The average response time to the aperiodic requests is similar to the Deferrable Server
 - ▶ The impact on the schedulability of the taskset is similar to that of a normal periodic task, as with the Polling Server
 - ▶ Replacing the servers for equivalent periodic tasks in the schedulability analysis (n is the total number of tasks, including servers)

$$U \leq n(2^{1/n} - 1)$$

Total Bandwidth Server

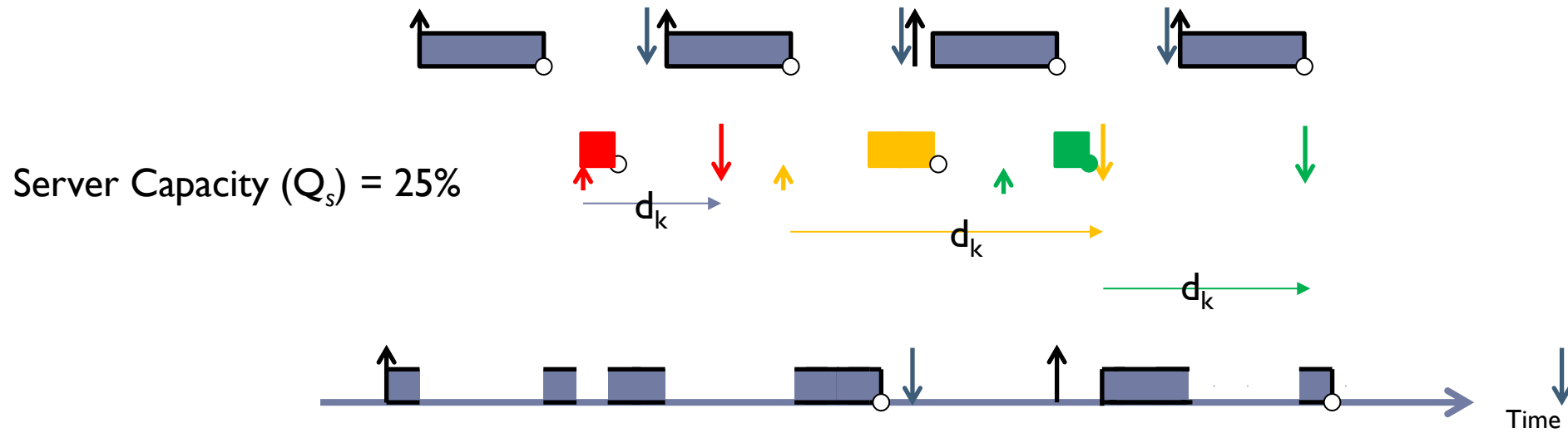
- ▶ The Total Bandwidth Server (TBS) is a dynamic priority server
 - ▶ Purpose is to handle aperiodic requests in EDF as early as possible
 - ▶ Maintaining a bandwidth to limit the impact on the taskset
 - ▶ When a new request (job) with c_k execution time arrives at instant t_k , a deadline d_k , is assigned to the server with utilization U_s

$$d_k = \max\{t_k, d_{k-1}\} + \frac{c_k}{U_s}$$

- ▶ Where d_{k-1} is the deadline of the previous job of the server, or zero if none
- ▶ We actually do not need to calculate server period and capacity, just utilization (U_s)

Total Bandwidth Server

- ▶ The Total Bandwidth Server (TBS) is a dynamic priority server



Total Bandwidth Server

- ▶ The Total Bandwidth Server (TBS) is a dynamic priority server
 - ▶ Complexity of implementation is relatively small since it is only necessary to calculate the deadline when a request arrives
 - ▶ The average response time is shorter than with dynamic priorities versions of the fixed-priority servers
 - ▶ The impact on the schedulability of the taskset is similar to that of a periodic task with the same bandwidth as that of the server: $U_p + U_s \leq 1$
- ▶ However
 - ▶ TBS requires that we know each job c_k *before execution*
 - ▶ TBS is vulnerable to overruns since there is no control over the actual execution time

Constant Bandwidth Server

- ▶ The Constant Bandwidth Server (CBS) is a dynamic priority server designed to solve the problem of TBS, enforcing bandwidth isolation
 - ▶ This is achieved using a capacity management scheme (Q_s, T_s) .
 - ▶ When a request arrives at instant t_k , and the server is idle, the deadline d_s of the server is computed as:

If $t_k + \frac{c_s}{U_s} \leq d_s^{current}$ then keep the same d_s (c_s is the currently remaining capacity)

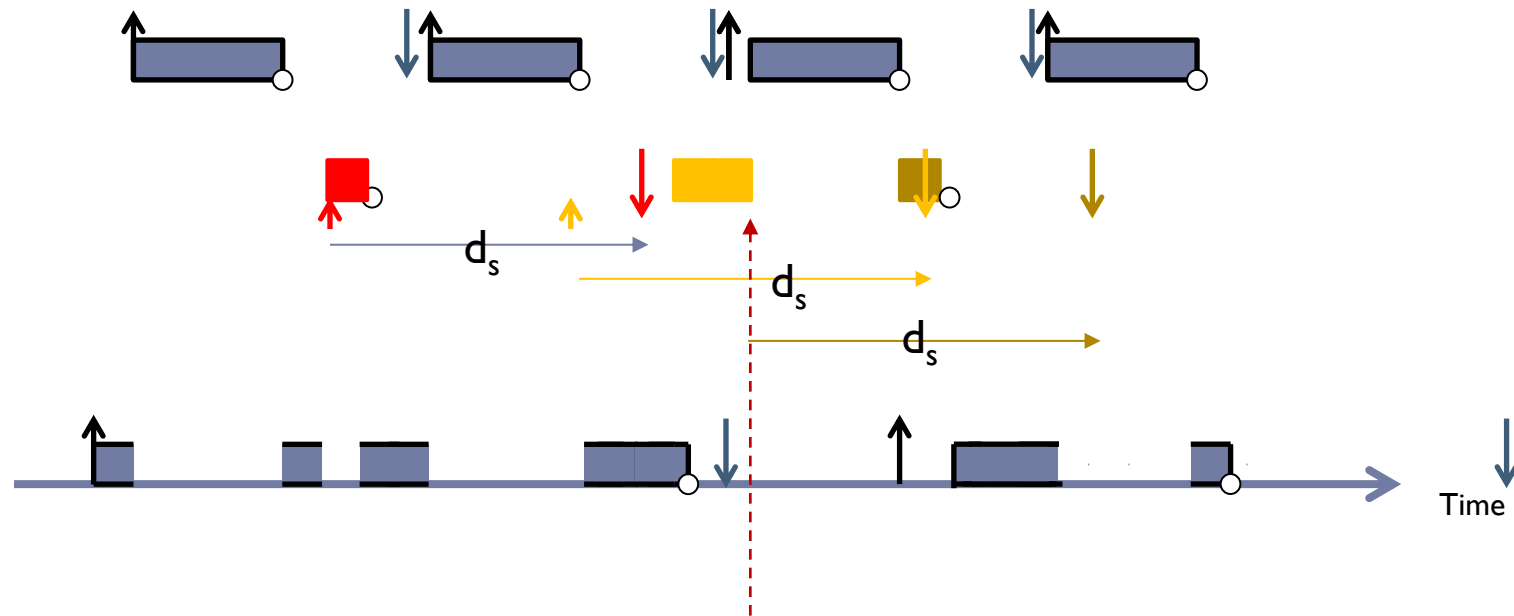
Else $d_s = t_k + T_s$ and $c_s = Q_s$

- ▶ If the server is not idle, the job is queued and will execute back-to-back with the current job
- ▶ When the server capacity is exhausted ($c_s = 0$) it is immediately replenished but d_s is postponed to enforce the server bandwidth

$d_s = d_s + T_s$ and $c_s = Q_s$

Constant Bandwidth Server

- ▶ The Constant Bandwidth Server (CBS) is a dynamic priority server designed to solve the problem of TBS, enforcing bandwidth isolation



Job did not finish and capacity is exhausted – deadline is postponed

Constant Bandwidth Server

- ▶ The Constant Bandwidth Server (CBS) is a dynamic priority server designed to solve the problem of TBS, enforcing bandwidth isolation
 - ▶ The deadline assignment scheme used by CBS enforces the server bandwidth independently of the aperiodic load that the server actually executes.
 - ▶ If a task executing inside a CBS executes for longer than expected, its deadline is automatically postponed, which in practice corresponds to lowering the task priority
 - ▶ The complexity of implementing a CBS is higher than that of the TBS due to the required capacity management.
 - ▶ There is still one single ready queue, for periodic and aperiodic tasks, managed by deadlines
 - ▶ The average response time to aperiodic requests is similar to that of the TBS.
 - ▶ The impact on the schedulability of the periodic subsystem is equal to that of a task with the same parameters than the server: $U_p + U_s \leq 1$

Constant Bandwidth Server

- ▶ The main motivation to use a CBS resides in the bandwidth isolation that it provides.
- ▶ A CBS can be used for:
 - ▶ Protecting a system of tasks overruns
 - ▶ Guaranteeing a minimal service to soft real-time tasks
 - ▶ Reserve bandwidth for any activity
- ▶ An alternative CBS (Hard CBS) is that when expiring budget, the task is suspended until the next period
 - ▶ And only at this point budget deadline is postponed and budget is replenished
- ▶ Linux implements a more complex variant of CBS